Protocolo individual

Estudiante: Jesús David Díaz Palencia | Código: 7502620080
Asignatura: Fundamentos de Programación (Unidad 1) | Carrera: Ingeniería de Software
Institución: Universidad de Cartagena — CTEV (2026-2)

Descripción del texto o actividad a realizar.

El presente protocolo individual sintetiza la lectura crítica, el análisis conceptual y la apropiación temática de la Unidad 1 del curso Fundamentos de Programación (Algoritmos y Programación Básica) de la Universidad de Cartagena, titulada 'Fundamentos de Programación y Diseño Algorítmico'. El estudio aborda el ciclo completo de desarrollo algorítmico: desde la descomposición abstracta del problema hasta la representación formal mediante pseudocódigo, diagramas de flujo normalizados (ANSI/ISO), tipado en memoria, operadores lógicos y aritméticos, estructuras de control (selección e iteración) y la verificación manual mediante pruebas de escritorio e implementación simulada en PSeInt.

Palabras claves.

Pensamiento Algorítmico, Pseudocódigo ANSI/ISO, Prueba de Escritorio, Tipos Primitivos de Datos, Operadores de Cortocircuito, Precedencia de Operadores, Asignación vs Igualdad, Estructuras de Control (Si-Entonces, Según, Para, Mientras, Repetir), PSeInt, Modularización.

Objetivos de las lecturas o actividad a realizar.

- Comprender las etapas del ciclo algorítmico para separar el análisis lógico y abstracto de la sintaxis del lenguaje de programación.
- Dominar la representación formal de algoritmos mediante pseudocódigo estructurado y diagramación de flujo normalizada.
- Analizar el comportamiento de los tipos primitivos en memoria y el orden estricto de evaluación de operadores aritmético-lógicos.
- Establecer una metodología rigurosa para validar la lógica algorítmica mediante pruebas de escritorio manuales antes de la codificación.
- Aplicar los conceptos teóricos construyendo y simulando 4 retos prácticos en PSeInt (Validador de Clave, Tabla de Multiplicar, Cajero Automático y Contador Par/Impar).

Conceptos claves y definiciones

Algoritmo: Secuencia ordenada, finita e inequívoca de pasos lógicos diseñada para procesar datos de entrada (input) y producir un resultado determinista (output).

Pseudocódigo y Diagramas ANSI/ISO: Lenguajes intermedios de especificación sintáctica y gráfica que representan el flujo lógico de un algoritmo de forma independiente a la plataforma o lenguaje formal.

Prueba de Escritorio: Simulación manual y paso a paso del estado de las variables sobre

papel o tabla de seguimiento, que permite verificar la exactitud lógica antes de compilar.

Tipos Primitivos y Memoria: Clasificación de los datos escalares (Enteros, Reales, Cadenas, Booleanos) que define su rango numérico, representación en memoria volátil y operaciones permitidas.

Operadores de Cortocircuito (Y / O): Mecanismos lógicos evaluados de forma optimizada donde el segundo término solo se procesa si el primero no determina por sí solo el resultado final.

Asignación (<- / =) vs Igualdad (== / =): La asignación traslada un valor o expresión al contenedor de memoria de la izquierda; la igualdad compara si dos operandos poseen el mismo valor numérico o lógico.

Estructuras de Control (Selección e Iteración): Instrucciones que alteran el flujo secuencial por defecto: condicionales (Si-Entonces, Según) y bucles (Para/For, Mientras/While, Repetir/Repeat).

Resumen de la(as) lecturas

La Unidad 1 de Fundamentos de Programación establece los cimientos de la carrera de Ingeniería de Software. La lección principal demuestra que la mayor trampa en la formación profesional es intentar escribir código de inmediato en un IDE sin haber diseñado e inspeccionado primero la solución algorítmica en lápiz y papel.

En primer lugar, se estudia la manipulación de tipos de datos y operadores. Se analiza cómo la precedencia matemática y los operadores de cortocircuito garantizan un uso eficiente de recursos y evitan errores en tiempo de ejecución. Se enfatiza la diferencia crucial entre la asignación de valor a una variable y la comparación relacional.

Posteriormente, la lectura profundiza en las estructuras de control. Se contrasta el uso del bucle 'Para' (apropiado para iteraciones contadas conocidas a priori) frente a los bucles 'Mientras' y 'Repetir-Hasta Que' (cuya ejecución depende de condiciones lógicas dinámicas).

Finalmente, se integra la validación práctica. Se demuestra que la combinación de pseudocódigo limpio, diagramación estandarizada y pruebas de escritorio exhaustivas constituye el estándar indispensable para desarrollar código mantenible, escalable y libre de fallos de diseño.

Metodología de trabajo (Cómo realizó la actividad)

La elaboración del presente protocolo individual se realizó siguiendo una metodología de aprendizaje autorregulado estructurada en tres fases:

- Fase 1. Lectura Analítica y Conceptualización: Estudio detallado de la Unidad 1 del módulo institucional y bibliografía complementaria, abstrayendo los conceptos de lógica algorítmica y control de flujo.
- Fase 2. Experimentación y Prueba de Escritorio en PSeInt: Construcción y validación

de 4 algoritmos prácticos (Validador de Clave, Tabla de Multiplicar, Cajero Automático y Contador Par/Impar) comprobando el estado de variables mediante trazado manual y ejecución en PSeInt.

• Fase 3. Síntesis y Redacción Académica: Consolidación de hallazgos en la plantilla institucional de la Universidad de Cartagena con rigor técnico de nivel de Ingeniería de Software.

Conclusiones de la lectura o actividad.

• El pensamiento algorítmico y la abstracción previa son independientes del lenguaje formal; un diseño lógico sólido se puede implementar fácilmente en Java, C++ o Python.

• La prueba de escritorio manual es la herramienta más eficaz para detectar bucles infinitos, desbordamientos y errores de lógica antes de la etapa de compilación.

• La selección precisa de la estructura iterativa (Para vs Mientras vs Repetir) responde al conocimiento estático o dinámico del número de repeticiones requeridas.

• El rigor en la validación de datos de entrada (programación defensiva) evita fallos de ejecución y garantiza la robustez del sistema.

Discusiones y recomendaciones.

A partir del análisis realizado, se formulan las siguientes recomendaciones para el proceso formativo:

• Institucionalizar el uso de pruebas de escritorio cruzadas entre compañeros antes de la entrega o codificación de proyectos.

• Reforzar en las sesiones de tutoría el diseño de diagramas de flujo estandarizados (ANSI/ISO) para visualizar condicionales anidadas complejas.

• Mantener una transición gradual y fundamentada desde la lógica pura en PSeInt hacia lenguajes con tipado estricto como Java o C++.

Bibliografía.

• Benavides Acuña, A. (2026). Módulo Unidad 1: Fundamentos de Programación y Diseño Algorítmico. Universidad de Cartagena.

• Dobrushkin, V. A. (2012). Métodos para análise de algoritmos. Rio de Janeiro: LTC.

• Joyanes Aguilar, L. (2020). Fundamentos de programación: algoritmos, estructura de datos y objetos (5.ª ed.). McGraw-Hill.

• Wanumen Silva, L. F., et al. (2017). Java básico. Ecoe Ediciones.

• PSeInt Software. (2025). Herramienta de desarrollo y simulación de pseudocódigo. https://pseint.sourceforge.net/

*** Nota**: La entrega del protocolo individual debe hacerse en formato **PDF**, a menos que el profesor indique otro tipo de archivo

Instructivo para diligenciar esta plantilla aquí